Homework 4 - Reinforcement Learning

ECE 157/272B TAs

Winter 2025

Due: Friday, March 14, 2025 by 11:59 PM
Log into the course GradeScope through the link on Canvas

Introduction

In this homework 4, we will provide practical experience in the last area of deep learning we have yet to cover: reinforcement learning. We will be using Gymnasium, a maintained fork of OpenAI's Gym library to train an agent to play the game of Lunar Lander. We will be using the DQN algorithm to train the agent.

Background

Reinforcement learning is a type of machine learning that involves training an agent to interact with an environment in order to maximize some notion of cumulative reward. The agent learns to achieve a goal in an uncertain, potentially complex environment. In reinforcement learning, an agent is given observations and, based on those, takes actions within an environment. In return it receives rewards. Its objective is to learn to act in a way that will maximize its expected long-term rewards – in other words, to learn a policy that maps observations to actions in a way that will maximize the expected sum of rewards. If we choose our rewards correctly, the agent will learn to achieve the goal we set for it. In Lunar Lander, that means to land the lander safely on the landing pad.

Environment

Unlike previous forms of machine learning, here we don't work with a dataset. Instead, the agent interacts with an environment, here defined in code as a simulation, though the environment may be a real place in the real world so long as it and the agent can be reset on success or failure. The environment we will be using is defined by the Gymnasium package: Lunar Lander v3. This is a 2D simulation of an abstract spacecraft landing on a landing pad of some airless world. The lander is subject to gravity and thrust, and must be controlled to land on the pad using its two orientation engines and one landing engine.

At every timestep, the environment returns an observation of the state of the lander, and the agent must return an action to take. The environment then advances to the next timestep, and the process repeats. The agent receives a reward at every timestep, and the goal is to learn a policy that maximizes the expected sum of rewards.

The space of observations is a simple one:

Observation Description	Range
The lander's position in x (with the landing pad center at 0)	$[-1.5, 1.5]$
The lander's position in y (with the landing pad center at 0)	$[-1.5, 1.5]$
The lander's velocity in x	$[-5.0, 5.0]$
The lander's velocity in y	$[-5.0, 5.0]$
The lander's orientation angle	$[-\pi, \pi]$
The lander's angular velocity	$[-5.0, 5.0]$
A boolean for if the left landing leg is touching the ground	$\{0, 1\}$
A boolean for if the right landing leg is touching the ground	$\{0, 1\}$

The space of actions is even simpler:

Action Description	Value
Do nothing	0
Fire left orientation engine	1
Fire main engine	2
Fire right orientation engine	3

Every step, the lander maps the given observations into the expected future reward for taking each action. The goal is to learn to estimate these future rewards well enough for all cases that the agent can always take the reward-maximizing action and perform well.

Tasks

Code

- The provided code trains a DQN agent to play the **CartPole-v1** environment. (You may run it through to get a trained CartPole model, though you are not required to do so.) Modify the code to train the agent to play the **LunarLander-v3** environment instead. You will need to change the `GYM_ENV` specified by the code and read through to make sure the code is compatible with the new environment's space of observations and actions.
- The provided code trains the model without any tracking of how it's performing beyond the duration of each episode. Modify the code to add a **gymnasium.wrappers.RecordVideo** wrapper around the environment used.
  - Your code should record a video of the agent playing the game every 25 episodes, and save the videos to the given `VIDEOS_DIR` directory. You don't need to worry about recording on specific episode numbers – as long as you record at least once within the first 50 episodes, and every 25 episodes after that, you've taken enough recordings to show the agent's progress. (Note: The environment counts episode numbers from 0.)
- Use your modified code to train a DQN agent to play the Lunar Lander environment. Train the agent for at least 600 episodes, to make sure it can perform the task consistently in a variety of starting conditions. You may train for more episodes if you wish, but you are not required to do so.
  - The provided hyperparameters should work for both the CartPole and Lunar Lander environments for a passing grade, but you may wish to adjust them to improve performance.
  - An agent is considered passing if it achieves over 200 points of reward in an episode. A consistently passing agent is one with an average reward over 200 points over the last 100 episodes. If your agent doesn't appear to be reaching that threshold, you may need to restart with a fresh model to try another set of hyperparameters and trajectories to learn from.
- The provided evaluation code only tracks the duration of episodes, not the reward achieved in each episode, because duration is all the CartPole environment requires. Modify the evaluation code to track the reward achieved in each episode.

- Evaluate your trained agent by running it for at least 100 episodes and checking the mean reward achieved. If the mean reward is over 200, your agent is passing. If not, you may need to train a new agent. When you have a passing agent, remember to download it!

Report items

1. The provided code uses a "Replay Memory" to store the agent's experiences and learn from that. **What is the purpose of the Replay Memory?** Why is it important to use a Replay Memory instead of just learning from the agent's most recent experiences?

2. **Submit the hyperparameters used in your final model.** Also make a note if you had other training attempts that didn't work out, though you are not required to submit their hyperparameters.

3. **Submit the training graphs for your final model.** These should show the reward and durations achieved in each episode. If you had other training attempts that didn't work out, you may include their graphs as well, though you are not required to do so. Looking at your final model's training graph, **Mark the eras where the model appears to have differing behaviors.** (E.g. if its average is slowly increasing, then plateaus, or if it's smooth then starts bouncing up and down, sometimes failing.) **About how many eras do you see? How long did it take the model to first really learn to play the game?**

4. Review the videos of your agent playing the game.

   (a) **List behaviors that you see, and when they appear. (Strategies, mistakes. . .)**

   (b) **Does the agent eventually learn to play the game well, or is it too reckless or too cautious?**

   (c) **Do the appearance and disappearance of behaviors in the videos match the eras you marked in the training graph?**

5. **Submit the evaluation duration and reward histograms of your final model, with their means marked. Does the model's policy appear to be consistent, or does it still struggle with the random starting conditions?**

What to turn in

- For the report, use the given slide template to make *slides* that provide the answers requested in the above Report items section. Export and submit the slides as a PDF document to GradeScope.

- Submit your code in notebook format (`.ipynb`) and your final lunar lander control network (`Lunar Lander-v3_dqn.pt`) to the GradeScope autograder.

  – Be sure to check that the autograder is able to successfully load and run your model. The provided model saving code transfers the model back to the CPU and saves it out with the correct name for the autograder, so you should not need to modify that code.